NAME: C. PRIYANKA

REG NO: 192511074

21) Prolog Program to Implement Towers of Hanoi

Aim

To write a Prolog program to solve the Towers of Hanoi problem using recursion.

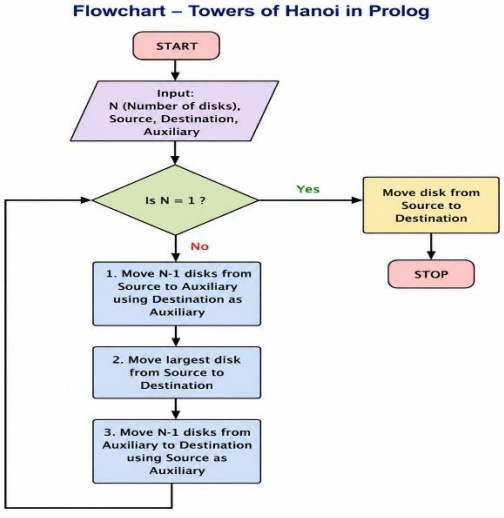
Objectives

- To understand recursion in Prolog.
- To move disks from one peg to another.
- To follow the rule that a larger disk cannot be placed on a smaller disk

Algorithm

1. If there is only one disk, move it directly from source to destination.
2. For N disks:
3. Move N-1 disks from source to auxiliary.
4. Move the largest disk from source to destination.
5. Move N-1 disks from auxiliary to destination.
6. Repeat until all disks are moved.

Flow Chart

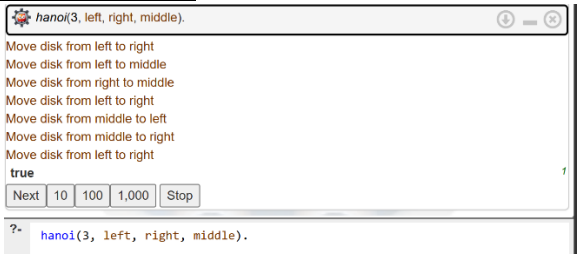


NKA (192511074)

Prolog Code

```
hanoi(1, Source, Destination, _) :-
    write('Move disk from '), write(Source),
    write(' to '), write(Destination), nl.

hanoi(N, Source, Destination, Auxiliary) :-
    N > 1,
    M is N - 1,
    hanoi(M, Source, Auxiliary, Destination),
    write('Move disk from '), write(Source),
    write(' to '), write(Destination), nl,
    hanoi(M, Auxiliary, Destination, Source).
```

Sample Output



C. PRIYANKA (19251074)

Result

The Towers of Hanoi problem was successfully implemented in Prolog using recursion.

Conclusion

Thus, the Prolog program successfully demonstrates recursive problem solving using the Towers of Hanoi problem.

22) Prolog Program to Check Whether a Bird Can Fly or Not

Aim

To write a Prolog program to determine whether a particular bird can fly or not using facts and rules.

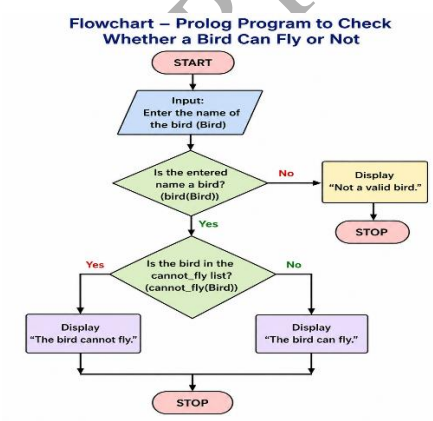
Objectives

- To understand facts and rules in Prolog.
- To represent bird characteristics.
- To use queries for determining flying ability.

Algorithm

1. Define birds as facts.
2. Define birds that cannot fly.
3. Create a rule can_fly/1.
4. Query the bird name.
5. Display whether the bird can fly or cannot fly..

Flow Chart

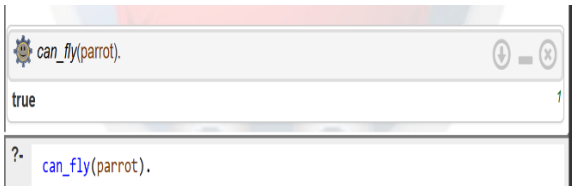


PRIYANKA (192511074)

Prolog Program

```
bird(parrot).
bird(eagle).
bird(pigeon).
bird(ostrich).
bird(penguin).

cannot_fly(ostrich).
cannot_fly(penguin).

can_fly(X) :-
   bird(X),
   \+ cannot_fly(X).
```

Sample Output



Result

The Prolog program successfully determines whether a given bird can fly or not.

Conclusion

Thus, the program demonstrates the use of **facts, rules, negation and queries** in Prolog.

23) Prolog Program to Implement Family Tree

Aim

To write a Prolog program to represent a family tree and find relationships between family members.

Objectives

- To represent family relationships using facts.
- To define parent, father, mother, brother and sister relationships.
- To understand logical inference in Prolog.

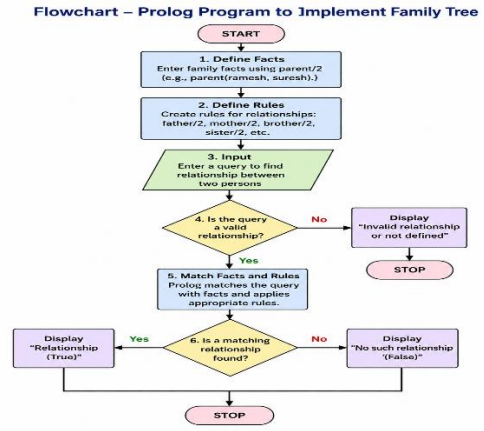
Algorithm

1. Define male and female members.

C. PRIYANKA (19251074)

2. Define parent relationships.
3. Create rules for father and mother.
4. Create rules for brother and sister.
5. Query the required relationship.

Flow Chart



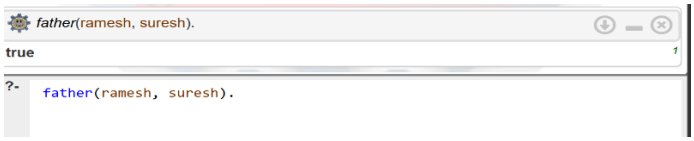
C. PRIYANKA (192511074)

Prolog Code

```
male(ramesh).
male(suresh).
male(rajesh).
male(rahul).

female(lakshmi).
female(sita).
female(priya).

parent(ramesh, suresh).
parent(lakshmi, suresh).
parent(ramesh, sita).
parent(lakshmi, sita).
parent(suresh, rahul).
parent(sita, priya).

father(X, Y) :-
    male(X),
    parent(X, Y).

mother(X, Y) :-
    female(X),
    parent(X, Y).

brother(X, Y) :-
    male(X),
    parent(P, X),
    parent(P, Y),
    X \= Y.

sister(X, Y) :-
    female(X),
    parent(P, X),
    parent(P, Y),
    X \= Y.
```

C. PRIYANKA (19251074)

Sample Output



Result

The family tree was successfully implemented using Prolog facts and rules.

Conclusion

Thus, Prolog can effectively represent family relationships and automatically infer relationships using logical rules.

24) Prolog Program to Suggest Dieting System Based on Disease

Aim

To write a Prolog program that suggests suitable food items based on a person's disease.

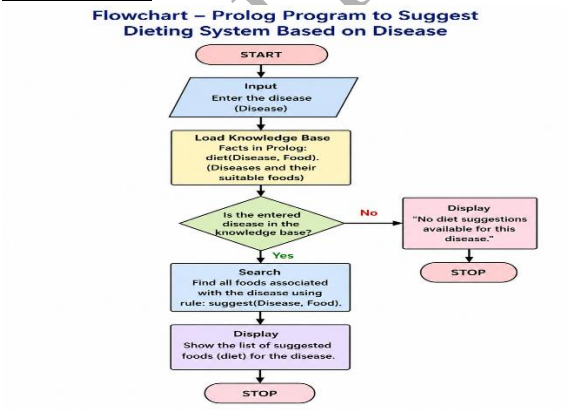
Objectives

- To represent diseases and suitable foods.
- To use rules for diet recommendation.
- To provide diet suggestions through Prolog queries.

Algorithm

1. Define different diseases as facts.
2. Define suitable food items for each disease.
3. Create a suggest/2 rule.
4. Enter the disease as a query.
5. Display the recommended food.

FlowChart

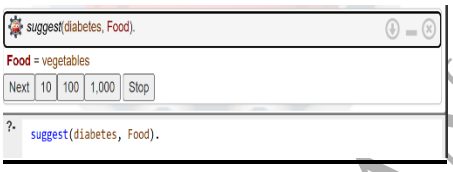


TYANKA (192511074)

Prolog Code

```
diet(diabetes, vegetables).
diet(diabetes, whole_grains).
diet(diabetes, fruits).

diet(hypertension, fruits).
diet(hypertension, vegetables).
diet(hypertension, low_salt_food).

diet(obesity, vegetables).
diet(obesity, salads).
diet(obesity, whole_grains).

suggest(Disease, Food) :-
    diet(Disease, Food).
```

Sample Output



Result

The Prolog program successfully provides diet suggestions based on the given disease.

Conclusion

Thus, Prolog can be used to develop a simple **rule-based diet recommendation system** using facts and logical rules.

C. PRIYANKA (192511074)